# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO1 ASSESSMENT TOOL 1 – Case Study Analysis

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO1 | **Assessment:** | Assessment Tool 1 – Case Study Analysis |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO1 Statement:** | *Apply advanced methodologies and agile project management techniques to manage software projects effectively* | | |
| **Student Name:** | U.Jayasudha | **Reg. No.:** | 192411228 |
| **Date:** | 05-10-26 | **Duration:** | 60 minutes |

**Code of Conduct**

*I U.Jayasudha(192411228) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: U.Jayasudha

## Annexure A – Case Study Analysis

**Instructions to Students:**

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

## Rubrics for Calculation

| Criteria | Excellent (5 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| Problem Understanding (30%) | Clearly explains the problem, objectives, and scope with complete understanding. | Explains the problem with minor omissions. | Basic understanding with limited explanation. | Poor understanding; objectives and scope are unclear or missing. |
| Stakeholder & Challenge Analysis (25%) | Identifies all stakeholders, their roles, and challenges with clear analysis. | Identifies most stakeholders and challenges with adequate analysis. | Identifies only a few stakeholders or challenges. | Stakeholders and system analysis are incomplete or inaccurate. |
| Current System Analysis (20%) | Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps. | Good analysis with most strengths and weaknesses identified. | Limited analysis with partial identification of issues. | Proposed solution is unclear or inappropriate; requirements are largely missing. |
| Proposed Software Solution & Software Engineering Principles (15%) | Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.). | Suitable solution with good application of software engineering principles. | Basic solution with limited application of software engineering principles. | Methodology is inappropriate, poorly justified, or not discussed. |

| | | | | |
|---|---|---|---|---|
| Methodology/ Framework Justification (10%) | Selects the most suitable SDLC/model/framework and provides strong technical justification. | Suitable methodology selected with adequate justification. | Methodology selected but justification is weak. | Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity. |

| Criteria | **Mark** |
|---|---|
| Problem Understanding (30%) | /5 |
| Stakeholder & Challenge Analysis (25%) | /5 |
| Current System Analysis (20%) | /5 |
| Proposed Software Solution & Software Engineering Principles (15%) | /5 |
| Methodology/ Framework Justification (10%) | /5 |
| **TOTAL** | /25 |

# CSA1010 – SOFTWARE ENGINEERING
# CASE STUDY ANALYSIS
## ASSIGNED QUESTION AND SOLUTION

## Problem Statement

"Online Examination Management System: Develop a platform for creating question banks, scheduling examinations, conducting online tests, automatically evaluating objective questions, and generating results."

## 1. Understand the Problem Statement

The proposed Online Examination Management System is a software platform that digitizes the process of conducting examinations. Faculty members can create and manage question banks, schedule examinations, and make examinations available to students through an online platform.

Students can log in, view scheduled examinations, answer objective questions, and submit their examination. The system automatically evaluates the answers, calculates marks, and generates results.

The central problem is the time-consuming and error-prone nature of traditional examination processes. Question preparation, examination scheduling, answer evaluation, mark calculation, and result preparation are often handled manually. The proposed system therefore connects question-bank management, examination scheduling, online testing, automatic evaluation, and result generation in one centralized platform.

**Objectives of the Proposed System**

- Provide a simple online platform for conducting examinations.
- Allow faculty to create and manage question banks.
- Allow administrators and faculty to schedule examinations.
- Allow students to attend examinations online.
- Automatically evaluate objective-type questions.
- Calculate marks accurately.
- Generate examination results quickly.
- Reduce manual work and calculation errors.
- Maintain centralized student and examination records.
- Provide administrators and faculty with examination reports.

**Scope of the System**

The scope covers student registration/login, faculty and administrator login, question-bank creation, question management, examination scheduling, online examination, answer submission, automatic evaluation, result generation, result viewing, and administrative management.

The system can be implemented as a responsive web application and later extended to mobile applications, online proctoring, AI-based question generation, performance analytics, and other advanced features. The initial version does not include advanced AI-based proctoring, biometric authentication, adaptive examinations, or complete offline examination support. These can be considered future enhancements.

## 2. Identify Key Stakeholders and Challenges

| Stakeholder | Role and Responsibilities | Major Challenges |
|---|---|---|
| Student | Logs in, views scheduled examinations, attends tests, submits answers, and views results. | Login problems, network issues, time management, technical difficulties. |
| Faculty | Creates questions, manages question bank, schedules examinations, and views student results. | Question preparation, managing large question banks, exam scheduling, result monitoring. |
| Administrator | Manages users, examinations, subjects, system settings, and reports. | Security, access control, data management, system availability. |
| System Administrator | Maintains database, server, backups, security, and system performance. | Server failures, data backup, security threats, system maintenance. |
| Management | Monitors examination performance and academic reports. | Limited consolidated information and difficulty identifying performance trends. |

**Key Challenges Across the System**

- Data accuracy: Incorrect questions, answers, or marks can affect examination results.
- System reliability: The examination system should remain available during examination periods.
- Security and privacy: Student information, questions, answers, and results must be protected.
- Performance: The system may need to support many students attending an examination simultaneously.
- Time management: Examination duration must be controlled automatically.
- Usability: Students and faculty require simple and role-specific interfaces.

- Connectivity: Internet problems may interrupt online examinations.
- Question management: Large question banks must be organized and maintained efficiently.

## 3. Analyze the Current Approach

### Current / Traditional Process

A typical traditional examination process begins when faculty prepare questions and create question papers. Students attend the examination and write their answers. After the examination, answer sheets are collected and evaluated manually. Faculty calculate marks and prepare the final results.

Question Preparation → Question Paper → Examination → Answer Collection → Manual Evaluation → Mark Calculation → Result Preparation

### Strengths of the Current Approach

- Existing faculty understand the examination process.
- Physical examinations can be conducted without internet access.
- Teachers can manually handle unusual cases.
- Direct interaction between students and faculty is possible.
- Physical records can provide evidence of examination participation.

### Limitations and Gaps

- Manual evaluation requires significant time.
- Manual mark calculation can create errors.
- Large amounts of paper may be required.
- Results may take several days to prepare.
- Question records may be distributed across files or paper documents.
- Maintaining previous examination records can be difficult.
- Students cannot receive immediate results.
- Managing examinations for large numbers of students is difficult.
- There is no centralized question-bank management.
- Repeated manual activities increase faculty workload.

## 4. Propose a Suitable Solution Using Software Engineering Principles

The proposed solution is a centralized, role-based Online Examination Management System accessible through a web application. Faculty can create questions and manage the question bank. Administrators can schedule examinations and manage users. Students can log in and attend available examinations. After submission, the system automatically evaluates objective questions and generates the result.

**Short Examination Flow**

**LOGIN → SELECT EXAM → ANSWER QUESTIONS → SUBMIT → AUTO EVALUATION → RESULT → END**

**High-Level Architecture**

| Presentation Layer | Application / Service Layer | Data Layer |
|---|---|---|
| Student Portal | Authentication & Role Management | Student Database |
| Faculty Dashboard | Question Bank Service | Question Database |
| Admin Dashboard | Examination Service | Examination Database |
| Online Exam Screen | Evaluation Service | Answer Database |
| Result Page | Result & Reporting Service | Result Database |

**Major Functional Modules**

**1. User Registration and Authentication**

Registration/login, password management, role-based access, and session management.

**2. Question Bank Management**

Faculty can add, edit, delete, organize, and manage objective questions and their correct answers.

**3. Examination Scheduling**

Faculty/admin can create an examination, select questions, set examination date, time, duration, and publish the examination.

**4. Online Examination**

Students can view questions, select answers, navigate through questions, track remaining time, and submit the examination.

**5. Automatic Evaluation**

The system compares submitted answers with the stored correct answers and automatically calculates the student's marks.

**6. Result Management**

The system generates examination results, displays marks and pass/fail status, and allows students to view their results.

**7. Administration and Reporting**

Administrators can manage users, examinations, subjects, records, and examination reports.

**Example Examination Status Flow**

**SCHEDULED → STARTED → IN PROGRESS → SUBMITTED → EVALUATED → RESULT GENERATED**

**Functional and Non-Functional Requirements**

| Functional Requirements | Non-Functional Requirements |
|---|---|
| 1. Users shall register/login according to their role. | 1. Usability: Simple and responsive interfaces. |
| 2. Faculty shall create and manage questions. | 2. Performance: Questions and results should load promptly. |
| 3. Faculty/admin shall schedule examinations. | 3. Reliability: Answers and results should not be lost. |
| 4. Students shall view available examinations. | 4. Security: Authentication and authorization must be applied. |
| 5. Students shall attend examinations online. | 5. Scalability: Support increasing students and examinations. |
| 6. The system shall record student answers. | 6. Maintainability: Modular and documented system. |
| 7. The system shall automatically evaluate answers. | 7. Availability: Minimize system interruptions. |
| 8. The system shall calculate marks. | 8. Compatibility: Support common browsers and devices. |
| 9. The system shall generate results. | |
| 10. Administrators shall manage users and examinations. | |

**Application of Software Engineering Principles**

**Modularity**

Separate question-bank, authentication, examination, evaluation, result, and administration modules are used.

**Scalability**

The architecture can support increasing numbers of students, questions, and examinations.

**Maintainability**

Modular design, coding standards, version control, logging, and automated testing make the system easier to maintain.

**Reliability**

Validation, database transactions, backups, controlled examination submission, and result verification help prevent data loss.

**Usability**

Role-specific dashboards, clear examination screens, readable questions, timers, and useful validation messages make the system easy to use.

**Security**

Role-based access control, strong authentication, secure password storage, input validation, encrypted communication, and audit logging protect examination information.

**Testability**

Individual modules can be tested using unit, integration, system, security, performance, and acceptance testing.

**Illustrative Database Entities**

User(UserID, Name, Email, Password, Role)

Question(QuestionID, Subject, QuestionText, OptionA, OptionB, OptionC, OptionD, CorrectAnswer)

Exam(ExamID, ExamName, Subject, ExamDate, ExamTime, Duration)

ExamQuestion(ExamID, QuestionID)

StudentAnswer(AnswerID, ExamID, StudentID, QuestionID, SelectedAnswer)

Result(ResultID, ExamID, StudentID, Marks, Status)

**Example Use Cases**

- Student attends an examination: Login → View Exam → Start Exam → Answer Questions → Submit → View Result.
- Faculty creates an examination: Login → Question Bank → Add Questions → Select Questions →

Set Date/Time → Publish Exam.

- Administrator manages users: Login → Dashboard → Add/Update User → Assign Role → Manage Account.
- Faculty views results: Login → Select Examination → View Student Results → Generate Report.

## 5. Justify the Chosen Methodology / Framework

**Selected Methodology: Agile Scrum**

Agile Scrum is suitable because the Online Examination Management System contains multiple user groups and workflows that can be developed and refined through feedback. Requirements such as examination interface design, question management, result reports, security, notifications, and future integrations may change after students and faculty use early versions. Scrum allows incremental development rather than waiting until the complete system is finished.

**Why Scrum is Appropriate**

- Incremental delivery: Login and question management can be delivered first, followed by examination, evaluation, and reporting.
- Frequent feedback: Students and faculty can review working modules and suggest improvements.
- Early risk identification: Security, performance, usability, and examination reliability issues can be identified early.
- Prioritization: Important features such as login, question bank, examination and evaluation can be developed first.
- Continuous testing: Every sprint can include unit, integration, security, usability, and acceptance testing.
- Adaptability: New requirements can be added to the product backlog and developed in future sprints.

**Suggested Scrum Team**

| Role | Responsibility |
|---|---|
| Product Owner | Prioritizes the backlog and represents student/faculty requirements. |
| Scrum Master | Facilitates Scrum activities and removes development obstacles. |
| Development Team | Designs, develops, tests, integrates, and documents the system. |
| Stakeholders | Provide feedback and validate whether the system |

| | |
|---|---|
| | meets examination needs. |

**Illustrative Sprint Plan**

| Sprint | Main Deliverables |
|---|---|
| Sprint 1 | Requirements, UI design, architecture, authentication, roles, database foundation. |
| Sprint 2 | Student/faculty registration and question-bank management. |
| Sprint 3 | Examination creation, scheduling, and online examination. |
| Sprint 4 | Automatic evaluation and result generation. |
| Sprint 5 | Admin dashboard, reports, security, and audit records. |
| Sprint 6 | Integration, performance, usability testing, bug fixing, and deployment preparation. |

**How Scrum Supports Maintenance**

After deployment, defects, security issues, performance improvements, and enhancement requests can be added to the product backlog and prioritized for future sprints. User feedback, monitoring, defect tracking, automated regression testing, and regular releases support continuous maintenance.

## 6. Discuss Expected Outcomes and Limitations

**Expected Outcomes and Benefits**

- Better examination management: Faculty can manage questions and examinations from one platform.
- Reduced manual work: Question management, evaluation, mark calculation, and result preparation are automated.
- Faster results: Objective questions can be evaluated immediately after submission.
- Improved accuracy: Automated evaluation reduces calculation mistakes.
- Centralized records: Questions, answers, examinations, and results are maintained in a centralized database.
- Better student experience: Students can attend examinations and view results online.
- Improved management visibility: Faculty and administrators can generate examination reports.
- Reduced paper usage: Online examinations reduce dependence on printed question papers and answer

sheets.

**How the Solution Addresses the Identified Challenges**

| Challenge | Proposed Response |
|---|---|
| Manual question management | Centralized question bank |
| Manual evaluation | Automatic objective-question evaluation |
| Calculation errors | Automated mark calculation |
| Delayed results | Immediate result generation |
| Security concerns | Authentication, role-based access and audit logs |
| Large number of students | Scalable system architecture |
| Record management | Centralized database |
| Examination time control | Automatic examination timer |
| Faculty workload | Automated examination and result processes |

**Potential Risks and Implementation Challenges**

- Internet connection failure may interrupt examinations.
- Server failure may affect examination availability.
- High student traffic may require performance scaling.
- Unauthorized access could expose examination information.
- Incorrect question or answer data may produce incorrect results.
- Students may face device or browser compatibility issues.
- Data migration from existing records may create data-quality problems.
- Users may require training before adopting the new system.

**Limitations of the Initial System**

- The system requires internet access for online examinations.
- Automatic evaluation is mainly suitable for objective questions.
- Technical problems can interrupt an examination.
- Advanced online proctoring may not be included in the first release.
- Security requires continuous monitoring and maintenance.
- Performance depends on server and network capacity.

**Future Enhancements**

- Mobile application for students and faculty.
- AI-assisted question generation.

- Random question selection.
- Randomization of answer options.
- Online examination proctoring.
- Face recognition for student authentication.
- AI-based cheating detection.
- Student performance analytics.
- Adaptive examinations.
- Cloud-based deployment.
- Email/SMS examination notifications.
- Advanced examination dashboards.

## Conclusion

The proposed Online Examination Management System provides a centralized software solution for creating question banks, scheduling examinations, conducting online tests, automatically evaluating objective questions, and generating results.

It addresses the major weaknesses of traditional examination processes by providing centralized question management, online examination, automatic evaluation, faster results, secure access, and organized examination records.

Agile Scrum is selected because the system can be developed incrementally and validated continuously with students, faculty, administrators, and other stakeholders. The proposed solution emphasizes modularity, scalability, maintainability, reliability, usability, security, and testability, providing a practical foundation for future growth.

**Rubric Alignment**

| **Rubric Area** | **Covered in This Solution** |
|---|---|
| Problem Understanding – 30% | Problem in own words, objectives, scope, inclusions and exclusions. |
| Stakeholder & Challenge Analysis – 25% | Students, faculty, administrators, system administrators and management; roles and challenges. |
| Current System Analysis – 20% | Traditional examination workflow, strengths, limitations and identified gaps. |
| Proposed Software Solution & Principles – 15% | Architecture, modules, requirements, examination flow, database entities, use cases and engineering |

| | |
|---|---|
| | principles. |
| Methodology / Framework – 10% | Agile Scrum selection, justification, roles, sprint plan, testing and maintenance approach. |